

Great! Here's a complete **Cloud Computing (CC) practical journal write-up** for a **Case Study on Amazon EC2 and its Web Services**, suitable for your LP-2 exam:

✓ **Practical Title:**

Case Study on Amazon EC2 and Learn About Amazon EC2 Web Services

✓ **Short Theory:**

Amazon EC2 (Elastic Compute Cloud) is a core service of Amazon Web Services (AWS) that provides scalable virtual servers in the cloud. EC2 enables users to run applications on virtual machines, known as **instances**, with customizable configurations including CPU, memory, storage, and networking.

It is widely used for web hosting, batch processing, analytics, and more. EC2 offers features like auto-scaling, elastic IP addresses, and various pricing models (on-demand, reserved, and spot instances).

✓ **Key Features of Amazon EC2:**

- **Elasticity:** Easily scale compute capacity up or down.
 - **Variety of Instance Types:** Supports general purpose, compute-optimized, memory-optimized, GPU, etc.
 - **Pay-as-you-go:** Charges based on usage.
 - **Secure:** Uses key pairs and security groups for secure access.
 - **Custom AMIs:** Create custom images to launch pre-configured instances.
 - **Load Balancing:** Integrated with AWS ELB (Elastic Load Balancing).
 - **Auto Scaling:** Automatically adjusts number of instances based on demand.
-

✓ **Steps to Launch an EC2 Instance:**

1. **Login to AWS Console** at <https://aws.amazon.com>
 2. Go to **EC2 Dashboard**
 3. Click **Launch Instance**
 4. Choose an **Amazon Machine Image (AMI)** (e.g., Ubuntu, Amazon Linux)
 5. Select an **instance type** (e.g., t2.micro – Free Tier eligible)
 6. Configure **instance details** (e.g., network, auto-assign IP)
 7. Add **storage** (default is 8 GB)
 8. Add **tags** (optional – e.g., Name = MyEC2Instance)
 9. Configure **security group** (open port 22 for SSH)
 10. Create/download a **key pair** (PEM file)
 11. Launch the instance
 12. Use a terminal or PuTTY to connect via SSH
-

✓ **Advantages of Amazon EC2:**

- High scalability and availability
 - Cost-effective with flexible pricing
 - Easily integrates with other AWS services
 - Secure and reliable infrastructure
 - Supports multiple operating systems and platforms
-

✓ **Requirements:**

- AWS Account
- Internet connection

- Basic knowledge of Linux/CLI (for SSH access)
-

✔ **Use Cases:**

- Hosting websites and web apps
 - Running batch processing and backend services
 - Deploying containers and microservices
 - Development and testing environments
-

✔ **Viva Questions & Answers:**

Q1: What is Amazon EC2?

A1: It's a web service that provides resizable compute capacity in the cloud.

Q2: What are EC2 instances?

A2: Virtual machines launched using Amazon EC2.

Q3: Name the EC2 pricing models.

A3: On-Demand, Reserved, Spot, and Savings Plans.

Q4: What is an AMI?

A4: Amazon Machine Image – a template used to launch EC2 instances.

Q5: What is a security group?

A5: A virtual firewall that controls inbound and outbound traffic to EC2 instances.

Q6: How do you access an EC2 instance?

A6: Via SSH using a private key (.pem) and the public IP address.

Would you like me to add a **diagram or screenshot steps** of the EC2 launch process as well?
40

You said:

This is cc practical 2 Installation and configure Google App

ChatGPT said:

Here's the complete write-up for your **Cloud Computing Practical 2:**

✔ **Practical Title:**

Installation and Configuration of Google App Engine (GAE)

✔ **Short Theory:**

Google App Engine (GAE) is a **Platform as a Service (PaaS)** offering by Google Cloud that allows developers to build and host web applications in Google-managed data centers. GAE automatically handles deployment, scaling, and load balancing, making it ideal for developers who want to focus on code instead of infrastructure.

GAE supports several languages including Python, Java, Node.js, PHP, and Go. Apps run in a secure, sandboxed environment and scale automatically based on traffic.

✔ **Features of Google App Engine:**

- Fully managed hosting platform
 - Automatic scaling and load balancing
 - Integrated versioning and rollback
 - In-built monitoring via Google Cloud Console
 - Supports popular programming languages
 - Integrates easily with Google Cloud services (e.g., Firestore, Cloud SQL)
-

✔ **Requirements:**

- Google Cloud Account
 - Internet connection
 - Google Cloud SDK installed
 - Basic knowledge of Python (or any supported language)
-

✅ **Steps to Install and Configure Google App Engine (Python Example):**

1. Create a Google Cloud Project:

- Go to <https://console.cloud.google.com>
- Click on **Create Project**
- Name your project and note the **Project ID**

2. Enable App Engine:

- Navigate to **App Engine > Dashboard**
- Click **Create Application**
- Select a region and preferred language (e.g., Python)

3. Install Google Cloud SDK:

- Download from <https://cloud.google.com/sdk>
- Install and run:

bash

CopyEdit

gcloud init

4. Create a Simple App (main.py):

python

CopyEdit

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')
def hello():
```

```
    return 'Hello from Google App Engine!'
```

5. Create app.yaml File:

yaml

CopyEdit

```
runtime: python39
```

```
entrypoint: gunicorn -b :$PORT main:app
```

```
handlers:
```

```
- url: /*
```

```
  script: auto
```

6. Deploy the Application:

- Run in terminal:

bash

CopyEdit

gcloud app deploy

- After deployment, access your app at:
<https://<project-id>.appspot.com>
-

✅ **Advantages:**

- No server management required
- High availability and scalability

- Easy to deploy and maintain
- Cost-effective for startups and small apps
- Secure and reliable (Google infrastructure)

✓ **Viva Questions & Answers:**

Q1: What is Google App Engine?

A1: It's a PaaS that lets developers host and scale web applications on Google Cloud.

Q2: Which programming languages does GAE support?

A2: Python, Java, Go, PHP, Node.js, Ruby, and .NET.

Q3: What is the purpose of app.yaml?

A3: It is a configuration file that defines the runtime and environment for the app.

Q4: What command is used to deploy an app on GAE?

A4: gcloud app deploy

Q5: What is the default hosting URL for deployed apps?

A5: https://<project-id>.appspot.com

Would you like a diagram showing the GAE architecture or deployment steps as well?

40

You said:

Creating an Application in Salesforce.com using Apex programming Language

ChatGPT said:

Here's your **Cloud Computing Practical: Creating an Application in Salesforce.com using Apex Programming Language:**

✓ **Practical Title:**

Creating an Application in Salesforce.com using Apex Programming Language

✓ **Short Theory:**

Salesforce.com is a cloud-based **CRM (Customer Relationship Management)** platform that provides tools for sales, service, marketing, and more. It allows developers to build custom applications using **Apex**, a proprietary, strongly typed, object-oriented programming language similar to Java.

Apex is hosted on the Force.com platform and tightly integrates with the Salesforce database and user interface, enabling developers to automate business logic and processes.

✓ **Features of Apex:**

- Similar syntax to Java
- Integrated with Salesforce data model
- Supports Triggers, Classes, and Visualforce pages
- Can be used to write custom business logic
- Runs in a multitenant environment with governor limits

✓ **Requirements:**

- Salesforce Developer Account (<https://developer.salesforce.com/>)
- Internet connection
- Basic understanding of objects, classes, and triggers
- Access to **Developer Console** or **VS Code with Salesforce Extension**

✓ **Steps to Create an Application in Salesforce using Apex:**

1. Sign up for a Salesforce Developer Account:

- Visit <https://developer.salesforce.com/signup>
- Fill in the details and activate your account via email

2. Log in to Salesforce and Navigate to Developer Console:

- Click on the avatar icon → **Developer Console**

3. Create a Custom Object (e.g., Student):

- Go to **Setup > Object Manager > Create > Custom Object**
- Enter label: Student, API name: Student
- Add fields like Name, Age, Course, etc.

4. Create an Apex Class:

- In Developer Console:
 - Go to **File > New > Apex Class**
 - Name it: StudentHandler
 - Example code:

apex

CopyEdit

```
public class StudentHandler {  
    public static void createStudent(String name, Integer age, String course) {  
        Student__c s = new Student__c();  
        s.Name = name;  
        s.Age__c = age;  
        s.Course__c = course;  
        insert s;  
    }  
}
```

5. Create an Apex Trigger (Optional):

- Go to **File > New > Apex Trigger**
- Trigger example on insert:

apex

CopyEdit

```
trigger StudentTrigger on Student__c (before insert) {  
    for(Student__c s : Trigger.new) {  
        if(s.Age__c < 18) {  
            s.Course__c = 'General Studies';  
        }  
    }  
}
```

6. Test the Application:

- Open **Execute Anonymous Window** in Developer Console:

apex

CopyEdit

```
StudentHandler.createStudent('Alice', 20, 'Computer Science');
```

- Check in the Salesforce UI under the **Student** object records.

✓ Advantages:

- Automates complex business processes
- Cloud-based, no installation needed
- Scalable and secure
- Custom logic and validations using Apex
- Integrated development environment

✓ **Viva Questions & Answers:**

Q1: What is Apex in Salesforce?

A1: Apex is a strongly typed, object-oriented programming language used to write custom logic on the Salesforce platform.

Q2: What is a trigger in Apex?

A2: A trigger is a piece of Apex code that executes before or after record manipulation (insert, update, delete).

Q3: How do you create custom objects in Salesforce?

A3: Go to Object Manager in Setup → Create → Custom Object → Define fields and settings.

Q4: What is the purpose of the Developer Console?

A4: It allows developers to write, run, and debug Apex code and monitor logs and performance.

Q5: Can we write SQL in Apex?

A5: No, but Apex uses **SOQL (Salesforce Object Query Language)** to query data from Salesforce objects.

Would you like a diagram or screenshot guide to include with this practical?

40

You said:

Design and develop custom Application (Mini Project) using Salesforce Cloud.

ChatGPT said:

Here is your **Cloud Computing Practical: Mini Project – Design and Develop a Custom Application using Salesforce Cloud:**

✓ **Practical Title:**

Design and Develop Custom Application (Mini Project) using Salesforce Cloud

✓ **Short Theory:**

Salesforce Cloud allows you to build and deploy custom applications without worrying about infrastructure. By using tools like **Salesforce Object Manager**, **Apex Programming**, **Lightning App Builder**, and **Visualforce**, you can create robust, secure, and scalable enterprise apps directly in the cloud.

This mini project demonstrates how to create a **Student Management System** with CRUD operations using **custom objects**, **Apex classes**, **Lightning pages**, and **workflow automation**.

✓ **Project Title:**

Student Management System on Salesforce

✓ **Objective:**

To manage student information such as Name, Age, Course, and Email with functionalities to add, update, and delete records and send email notifications on record creation.

✓ **Modules/Features:**

- Add New Student
- View/Edit/Delete Students
- Email notification on student creation

- Search students
- Custom Lightning App

✓ **Tools & Requirements:**

- Salesforce Developer Edition
 - Internet connection
 - Access to:
 - **Object Manager**
 - **Apex Classes & Triggers**
 - **Lightning App Builder**
 - **Process Builder / Flows**
-

✓ **Steps to Build the Application:**

1. Create a Custom Object:

- Go to **Setup > Object Manager > Create > Custom Object**
- Object Name: Student
- Fields:
 - Name (Standard)
 - Age (Number)
 - Course (Text)
 - Email (Email)

2. Create a Tab for Student:

- Setup > Tabs > Custom Object Tabs → Create new tab for Student

3. Create a Lightning App:

- Setup > App Manager > New Lightning App
- Name: Student Management
- Add the Student object tab to the app

4. Add Apex Class (Optional):

apex

CopyEdit

```
public class StudentService {
    public static void addStudent(String name, Integer age, String course, String email) {
        Student__c s = new Student__c(Name=name, Age__c=age, Course__c=course,
        Email__c=email);
        insert s;
    }
}
```

5. Add Trigger to Send Email on Creation:

apex

CopyEdit

```
trigger StudentTrigger on Student__c (after insert) {
    for(Student__c s : Trigger.new) {
        Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
        mail.setToAddresses(new String[] {s.Email__c});
        mail.setSubject('Welcome to the Course');
        mail.setPlainTextBody('Hi ' + s.Name + ',\n\nWelcome to ' + s.Course__c + ' course.');
```

6. Create a Flow or Process for Automation (Optional):

- Use **Flow Builder** to automate welcome messages or status updates.

7. Test the Application:

- Add some student records via the UI
- Check email notifications (to dummy emails or logs)
- Update/Delete records and observe changes

✓ Advantages:

- Full control over data and UI
- Fast cloud deployment
- Minimal coding using Salesforce tools
- Easy scalability and integrations
- In-built security and access control

✓ Viva Questions & Answers:

Q1: What are custom objects in Salesforce?

A1: Custom objects are user-defined objects to store custom data specific to the organization's requirements.

Q2: What is the difference between Apex Class and Trigger?

A2: Apex Class is used to define reusable logic, while a Trigger is used to automate actions based on data changes.

Q3: What tools did you use to build this application?

A3: Object Manager, Apex Classes, Triggers, Lightning App Builder, Flow Builder.

Q4: How does Salesforce handle email notifications?

A4: Through Apex Messaging class or automation tools like Process Builder and Flow.

Q5: Can this app be used on mobile?

A5: Yes, Salesforce Lightning apps are responsive and accessible via the Salesforce mobile app.